Chapter 6

Prof Dr. Abbas Khalid

Functions
Introduction

Built-in Functions

User-defined Functions
Function Prototype
Function Call
Function Definition

Passing Arguments to a Function

Conclusions

Programming Fundamentals (Functions)

Functions

Have you ever seen a function? $\rightsquigarrow$ *main*()

- The *main* function indicates the beginning of a C++ program and it must be included in every C++ program.

```
int main()                    ▷ main is name of the function.
    {                         ▷ beginning of the body of main
    ⋮                         ▷ statements executed by main
    return 0;                 ▷ main returns a value 0.
    }                         ▷ Closing of the body of main
```

- The parentheses after main indicate that main is a function.
- The word int to the left of main indicates that main returns an integer value upon its completion (see *return* 0; statement).
- Any thing inside the parentheses is (are) called parameter(s).

Functions (2)

- A program may consist of several hundred statements. If this program is written as one long program, it becomes difficult to handle it.
- To make the program simple, it is divided into smaller and independent modules. These modules are called functions.
- Any sequence of instructions that appear in a program more than once can be written as a function.
- A function code is stored in only one place in the memory even if the function is executed multiple times.
- A function is invoked by a function call. When a function is called, the code that the function contains is executed, when it is finished, execution of the program continues immediately after the point where the function was called.

Functions (3)

- Any function can call other functions that may in turn call other functions. So a single function call could result in several functions begin executed.
- To keep the track of memory location from where the call was made and where the function must return, a call stack is used.
- Each function is written to perform a specific task. It is written and tested independently.
- Since each function is written independently, several programmers can work on one program simultaneously, each doing his job independently.
- Once a function has been written and tested, it can also be used in other programs.
- Use of functions reduces over all length of the program.

Classification of functions

- In C++, functions can be classified into two types:
  1. User-defined functions
  2. Built-in (library) functions

- User-defined functions are created by the programmer as per the requirement of the program. These functions have to declared and defined before their use.
- Built-in functions are part of the compiler package. Such functions are used without declaring and defining them. To use built-in functions in a program, we only include the header file where the required function is defined.

Built-in Functions

Chapter 6

Prof Dr. Abbas Khalid

Functions
Introduction
Built-in Functions
User-defined Functions
Function Prototype
Function Call
Function Definition
Passing Arguments to a Function
Conclusions

1. **Name:** *abs*(*x*) **Header File:** < *cmath* >

   Use: Returns the absolute value of its argument.

   - **Parameter(s):** *int*(*double*) **Result:** *int*(*double*)
   - **Example:** *abs*(−7) = 7.

2. **Name:** *fabs*(*x*) **Header File:** < *cmath* >

   Use: Returns the absolute value of its argument.

   - **Parameter(s):** *double* **Result:** *double*
   - **Example:** *fabs*(−5.67) = 5.67.

3. **Name:** *ceil*(*x*) **Header File:** < *cmath* >

   Use: Returns the smallest whole number $\not<$ *x*.

   - **Parameter(s):** *double* **Result:** *double*
   - **Example:** *ceil*(56.34) = 57.0.

4. **Name:** *floor*(*x*) **Header File:** < *cmath* >

   Use: Returns the largest whole number $\not>$ *x*.

   - **Parameter(s):** *double* **Result:** *double*
   - **Example:** *floor*(45.67) = 45.00.

Built-in Functions

Chapter 6

Prof Dr. Abbas Khalid

Functions
Introduction
Built-in Functions
User-defined Functions
Function Prototype
Function Call
Function Definition
Passing Arguments to a Function
Conclusions

5. **Name:** $cos(x)$ **Header File:** $< cmath >$

   Use: Returns the cosine of angle x.

   - **Parameter(s):** *double* **Result:** *double*
   - **Example:** $cos(0.0) = 1.0$.

6. **Name:** $exp(x)$ **Header File:** $< cmath >$

   Use: Returns e^x, where $e = 2.718$.

   - **Parameter(s):** *double* **Result:** *double*
   - **Example:** $exp(1.0) = 2.718$.

7. **Name:** $pow(x, y)$ **Header File:** $< cmath >$

   Use: Returns x^y; if $x < 0$, y must be a whole number.

   - **Parameter(s):** *double* **Result:** *double*
   - **Example:** $pow(0.16, 0.5) = 0.4$.

8. **Name:** $sqrt(x)$ **Header File:** $< cmath >$

   Use: Returns the square root of x; x must be ≥ 0.

   - **Parameter(s):** *double* **Result:** *double*
   - **Example:** $sqrt(4.0) = 2.0$.

Built-in Functions

Chapter 6

Prof Dr. Abbas Khalid

Functions
Introduction
Built-in Functions
User-defined Functions
Function Prototype
Function Call
Function Definition
Passing Arguments to a Function
Conclusions

9. **Name:** *islower*(x) **Header File:** $<$ *cctype* $>$

   Use: Returns 1 (true) if x is a lowercase letter; otherwise, it returns 0 (false).

   - **Parameter(s):** *int* **Result:** *int*

10. **Name:** *isupper*(x) **Header File:** $<$ *cctype* $>$

    Use: Returns 1 (true) if x is an uppercase letter; otherwise, it returns 0 (false).

    - **Parameter(s):** *int* **Result:** *int*

11. **Name:** *tolower*(x) **Header File:** $<$ *cctype* $>$

    Use: Returns the lowercase value of x if x is uppercase; otherwise, it returns x.

    - **Parameter(s):** *int* **Result:** *int*

12. **Name:** *toupper*(x) **Header File:** $<$ *cctype* $>$

    Use: Returns the uppercase value of x if x is lowercase;otherwise, it returns x.

    - **Parameter(s):** *int* **Result:** *int*

Built-in Functions

Chapter 6

Prof Dr. Abbas Khalid

Functions
Introduction

Built-in Functions

User-defined Functions
Function Prototype
Function Call
Function Definition

Passing Arguments to a Function

Conclusions

Examples:

1. *cout* << "*abs*(−7) =" << *abs*(−7) << *endl*; ⇝ 7
2. *cout* << "*fabs*(−5.67) =" << *fabs*(−5.67) << *endl*; ⇝ 5.67
3. *cout* << "*ceil*(56.34) =" << *ceil*(56.34) << *endl*; ⇝57
4. *cout* << "*floor*(56.34) =" << *floor*(56.34) << *endl*; ⇝ 56
5. *cout* << "*cos*(0) =" << *cos*(0) << *endl*; ⇝ 1
6. *cout* << "*sin*(0) =" << *sin*(0) << *endl*; ⇝ 0
7. *cout* << "*islower*('A') =" << *islower*(65) << *endl*; ⇝0
8. *cout* << "*isupper*('A') =" << *isupper*('A') << *endl*; ⇝1
9. *cout* << "*tolower*('A') =" << *tolower*('A') << *endl*; ⇝ 97
10. *cout* << "*toupper*('A') =" << *toupper*('A') << *endl*; ⇝ 65

Built-in Functions

Chapter 6

Prof Dr. Abbas Khalid

Functions
Introduction
Built-in Functions
User-defined Functions
Function Prototype
Function Call
Function Definition
Passing Arguments to a Function
Conclusions

11 *cout* << "*pow*(2.0, 3.0) =" << *pow*(2.0, 3.0) << *endl*; $\rightsquigarrow$ 8

12 *cout* << "*sqrt*(9.0) =" << *sqrt*(9.0) << *endl*; $\rightsquigarrow$ 3

13 *cout* << "*exp*(0) =" << *exp*(0) << *endl*; $\rightsquigarrow$ 1

Exercise: Using built-in functions, write each of the following as a C++ expression.

1. $3.75^{6.8}$
2. $\sqrt{x-2y}$
3. $w^{\frac{t}{3}}$
4. $\frac{-b+\sqrt{b^2-4ac}}{2a}$
5. $|x+2y-3|$

User-defined Functions

Chapter 6

Prof Dr. Abbas Khalid

Functions
Introduction

Built-in Functions

User-defined Functions
Function Prototype
Function Call
Function Definition

Passing Arguments to a Function

Conclusions

In C++, three components are required for a function. These are:

1. Function prototype
2. Function call
3. Function definition

Function Prototype

Chapter 6

Prof Dr. Abbas Khalid

Functions
Introduction
Built-in Functions
User-defined Functions
Function Prototype
Function Call
Function Definition
Passing Arguments to a Function
Conclusions

- A function prototype tells the compiler the name of the function, the type of data returned by the function, the number of parameters, the types of parameters and the order in which these parameters are expected.
- The general form of a function prototype is:

  $$returnType\ functionName(parameter - list);$$

- Note that a function prototype is always terminated by a semicolon.
- The parameter-list specifics the type and number of parameters provided to the function. If more than one parameter is used, they are separated by commas. If the function has no parameter, then *void* is used in between the parentheses or these parentheses are left empty.

Function Prototype

Chapter 6

Prof Dr. Abbas Khalid

Functions
Introduction
Built-in Functions
User-defined Functions
Function Prototype
Function Call
Function Definition
Passing Arguments to a Function
Conclusions

- The *returnType* specifics the data type of the value to be returned by the function. This may be any valid data type e.g., *int*, *float* etc. If a function does not return any value, the keyword *void* is used.
- The portion of a function prototype that includes the name of the function and the type of its parameters is called function signature. The function signature does not include the return type of the function.
- A function prototype does not require parameter names in the parameter list. They require only types. If parameter names are used, the compiler ignores the names.
- A function prototype can be placed almost everywhere in a program. If it is placed outside any function, it applies to all calls to the function appearing after the function prototype in the file. A function prototype placed in a function applies only calls made in that function.

Function Prototype

Chapter 6

Prof Dr. Abbas Khalid

Functions
Introduction
Built-in Functions
User-defined Functions
Function Prototype
Function Call
Function Definition
Passing Arguments to a Function
Conclusions

Example: Write function prototypes for the functions with following specifications.

1. **Name:** show **No. of parameters:** None
   Returns: None **Parameter types:** None

Solution: *void show*(); OR *void show*(*void*);

2. **Name:** get **No. of parameters:** None
   Returns: *double* **Parameter types:** None

Solution: *double get*(); OR *double get*(*void*);

3. **Name:** set **No. of parameters:** 1
   Returns: None **Parameter types:** *double*

Solution: *void set*(*double*);

Function Prototype

4 **Name:** add **No. of parameters:** 2

Returns: *double* **Parameter types:** *double*, *int*

Solution: *double add*(*double*, *int*);

5 **Name:** write **No. of parameters:** 3

Returns: None **Parameter types:** *double*, *int*, *char*

Solution: *void write*(*double*, *int*, *char*);

Function Prototype

Exercise: Which of the following are correct function prototypes?

1. *void display*(x, y, u);
2. *int display*$($*int*, *double*, *char*$)$
3. *int display*$($*int*, *double*, *char*$)$;
4. *int display*$($*int* x, *double* y, *char* $u$$)$;
5. *display*$($*int*, *double*, *char*$)$;

Function Call

Chapter 6

Prof Dr. Abbas Khalid

Functions
Introduction
Built-in Functions
User-defined Functions
Function Prototype
Function Call
Function Definition
Passing Arguments to a Function
Conclusions

- A function is always called from another function [usually main()]. The general form for a function call that returns no value is:

  functionName(*argument – list*);

- The argument-list only contains the names of parameters to be passed to the function.
- The general form for a function call that returns a value is:

  dataType varName = *functionName*(*argument – list*);

- The *dataType* must be compatible with the *returnType* of the function. The variable *varName* will store the value returned by the function.

Function Call

Chapter 6

Prof Dr. Abbas Khalid

Functions
Introduction
Built-in Functions
User-defined Functions
Function Prototype
Function Call
Function Definition
Passing Arguments to a Function
Conclusions

Example: Write function calls for each of the following functions prototypes.

1. *void show*(); OR *void show*(*void*);

Solution: *show*();

2. *double get*(); OR *double get*(*void*);

Solution: *double value* = *get*();

3. *void set*(*double*);

Solution: *set*(3.5); OR *double x* = 3.5; *set*(*x*);

4. *double add*(*double*, *int*);

Solution: *double result* = *add*(3.5, 3);

5. *void write*(*double*, *int*, *char*);

Solution: *write*(3.5, 5, 'A');

Function Call

Chapter 6

Prof Dr. Abbas Khalid

Functions
Introduction

Built-in Functions

User-defined Functions
Function Prototype
Function Call
Function Definition

Passing Arguments to a Function

Conclusions

Exercise: Consider the following statements:

int $num1, num2, num3;$ $num1 = 6; num2 = 7; num3 = 4;$

double $x, w, h;$ $x = 6.2; w = 2.3; h = 3.4;$

- and the function prototype:

double $box($*double*, *double*, *double*$);$

- Which of the following function calls are valid?

1. *double* $volume = box(x, w, h);$
2. *double* $volume = box(x, 3.8, h);$
3. $cout << box(num1, num3, num2) << endl;$
4. $cout << box(x, w, 7.0) << endl;$
5. *double* $volume = box(x, num1, h);$
6. $cout << box(6.2, , h) << endl;$
7. *double* $volume = box(x + w, h);$
8. *double* $volume = box(num1, num2 + num3);$

Function Definition

Chapter 6

Prof Dr. Abbas Khalid

Functions
Introduction
Built-in Functions
User-defined Functions
Function Prototype
Function Call
Function Definition
Passing Arguments to a Function
Conclusions

- The general format for a function definition is:

```
returnType functionName(parameter-list){
        //body of function
        return value;
    }
```

- The first line in this format is called function header. It must match with the function prototype except parameter names and a semicolon.
- The declarations and statements in braces form the function body. A function body is also referred as a block. Variables declared here are local to this block only.

Function Definition

There are three ways to return control to the point at which the function is invoked:

- If the function does not return a result, omit the *return* statement. The control is returned simply when the function-ending brace is reached.
- An alternative approach to above situation is to execute the following statement:

  return;

  i.e., no value is specified along with *return* statement.
- If the function returns a result, then specify that value along with *return* statement i.e.,

  return value;

Note that if the definition of the function appears before the functions first use in the program, then function prototype is not required. In such a case, function definition also acts as the function prototype.

Function Definition

Example: Write function definitions for each of the following:

1. **Prototype:** *void show*(); OR *void show*(*void*);

   Call: *show*();

   Definition:

   void show()

   {

   ⋮

   return; ▷ optional when return type is *void*.

   }

2. **Prototype:** *void set*(*double*);

   Call: *set*(3.5);

   Definition:

   void set(*double r*){

   ⋮

   }

Function Definition

3. **Prototype:** *double get();* OR *double get(void);*

   Call: *double value = get();*

   Definition:

```
double get()
    {
      double result;
      :
       return result;
    }
```

Function Definition

4 **Prototype:** *double add(double, int);*

Call: *double result = add(3.5, 3);*

Definition:

```
double add(double x, int y)
    {
    double result = x + y;
     return result;
    }
```

Function Definition

Exercise: Consider the following function and write the output of the asked statements.

```
int discover(int one, int two)
{
  int secret;
  secret = (one >= two) ? one : two;
  return secret;
}
```

1. `int x = 12; int y = 8; cout << discover(y,x) << endl;`
2. `int x = 8; int y = 12; int z = discover(y,x); cout << z << endl;`
3. `cout << discover(10) << endl;`

Exercise 1: What will be the output?

Chapter 6

Prof Dr. Abbas Khalid

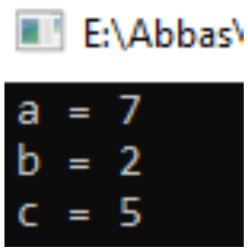
Functions
Introduction
Built-in Functions
User-defined Functions
Function Prototype
Function Call
Function Definition
Passing Arguments to a Function
Conclusions

```
#include<iostream>
using namespace std;

void test(int, int, int); // Prototype

int main()  {
    int a, b, c;
    a = 2; b = 5; c = 7;
    test(a,b,c);        // Call

return 0;
}
void test (int b, int c, int a) // Definition
    {
     cout<<" a = "<<a<<endl;
     cout<<" b = "<<b<<endl;
     cout<<" c = "<<c<<endl;
   }
```



Exercise 2: What will be the output?

Chapter 6

Prof Dr. Abbas Khalid

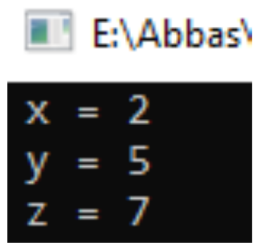
Functions
Introduction
Built-in Functions
User-defined Functions
Function Prototype
Function Call
Function Definition
Passing Arguments to a Function
Conclusions

```
#include<iostream>
using namespace std;

void test(int, int, int); // Prototype

int main()  {
    int a, b, c;
    a = 2; b = 5; c = 7;
    test(a,b,c);        // Call

return 0;
}
void test (int x, int y, int z) // Definition
    {
     cout<<" x = "<<x<<endl;
     cout<<" y = "<<y<<endl;
     cout<<" z = "<<z<<endl;
   }
```



Remarks

- It is a syntax error if the function prototype, function header and function call do not agree in the number, type and order of arguments. They must also agree in the type of return value, and function name.
- A function cannot be defined inside another function definition.
- There is difference between a parameter and an argument. A parameter appears in a function definition and specifies the data type expected by the function. An argument is the actual value passed to the function when the function is called.
- A function can return only a single value.

Programming Tasks

Chapter 6

Prof Dr. Abbas Khalid

Functions
Introduction
Built-in Functions
User-defined Functions
Function Prototype
Function Call
Function Definition
Passing Arguments to a Function
Conclusions

1. Write a function that returns the minimum of two integers.
2. Write a function that reads an input from user and returns the entered value to the calling program.
3. Write a function that prints "Even" or "Odd" depending on the value passed to that function.
4. Write a function which receives a *float* and an *int* from *main*, finds the product of these two and returns the product which is printed through *main*.
5. Write a function that receives 5 integers and returns the average of these numbers. Call this function from *main* and print the result in *main*.

Passing Arguments to a Function

Chapter 6

Prof Dr. Abbas Khalid

Functions
Introduction

Built-in Functions

User-defined Functions
Function Prototype
Function Call
Function Definition

Passing Arguments to a Function

Conclusions

There are two ways of passing arguments to a function:

1. Call by value
2. Call by reference

Call By Value:

1. Variables or constants specified as arguments are not actually passed to a function. Instead, the copies of the arguments are created and these copies are used as values to be transferred.
2. Once the execution of the function is complete, the copies of the arguments are discarded.

Call By Value

Chapter 6

Prof Dr. Abbas Khalid

Functions
Introduction
Built-in Functions
User-defined Functions
Function Prototype
Function Call
Function Definition
Passing Arguments to a Function
Conclusions

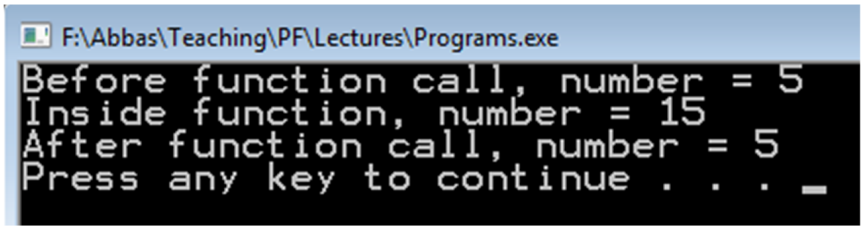
```
#include <iostream>
using namespace std;

void callByValue (int);
int main ()
   {

   int number = 5;
   cout<<"Before function call, number = "<<number<<endl;
   callByValue (number);
   cout<<"After function call, number = "<<number<<endl;

   return 0;
  }

void callByValue (int number)
  {
   number = number + 10;
   cout<<"Inside function, number = "<<number<<endl;
  }
```



Passing Arguments to a Function

Chapter 6

Prof Dr. Abbas Khalid

Functions
Introduction
Built-in Functions
User-defined Functions
Function Prototype
Function Call
Function Definition
Passing Arguments to a Function
Conclusions

Call By Reference:

1. A reference is simply an alias for another variable.
2. In call by reference mechanism, when a function is called, the reference parameter becomes an alias for the argument value in the calling program.
3. Whenever this parameter is used in the body of the function, it will access the argument value in the calling function directly. To specify a reference type just add & (ampersand) after the type e.g., *int&*.
4. The advantage of passing by reference is that the function can access the actual variables in the calling program. This provides a mechanism for returning more than one value from the function back to the calling program.

Call By Reference

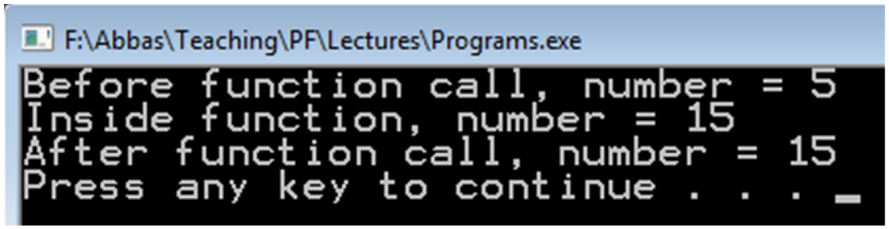

```
#include <iostream>
using namespace std;

void callByRef (int&);
int main ()
    {

    int number = 5;
    cout<<"Before function call, number = "<<number<<endl;
    callByRef (number);
    cout<<"After function call, number = "<<number<<endl;

    return 0;
   }

void callByRef (int& number)
   {
    number = number + 10;
    cout<<"Inside function, number = "<<number<<endl;
   }
```



Call By Reference

Chapter 6

Prof Dr. Abbas Khalid

Functions
Introduction
Built-in Functions
User-defined Functions
Function Prototype
Function Call
Function Definition
Passing Arguments to a Function
Conclusions

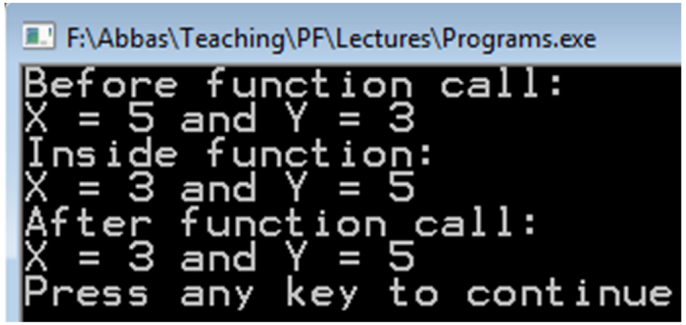
```
#include <iostream>
using namespace std;

void callByRef (int&, int& );
int main ()
   {
   int x = 5; int y = 3;

   cout<<"Before function call: "<<endl;
   cout<<"X = "<<x<<" and Y = "<<y<<endl;

   callByRef (x,y);
   cout<<"After function call: "<<endl;
   cout<<"X = "<<x<<" and Y = "<<y<<endl;
   return 0;
  }

void callByRef (int& a, int& b)
  {
   a = 3;  b = 5;
   cout<<"Inside function: "<<endl;
   cout<<"X = "<<a<<" and Y = "<<b<<endl;
  }
```



Programming Tasks

Chapter 6

Prof Dr. Abbas Khalid

Functions
Introduction

Built-in Functions

User-defined Functions
Function Prototype
Function Call
Function Definition

Passing Arguments to a Function

Conclusions

1. Write a function that receives 5 integers and returns the sum, average and standard deviation of these numbers. Call this function from *main* and print the results in *main*.
2. Write a function that receives marks received by a student in 3 subjects and returns the average and percentage of these marks. Call this function from *main* and print the results in *main*.

Conclusions

Chapter 6

Prof Dr. Abbas Khalid

Functions
Introduction

Built-in Functions

User-defined Functions
Function Prototype
Function Call
Function Definition

Passing Arguments to a Function

Conclusions

In this lecture we discussed:

- Built-in functions
- User-defined functions
- Function Prototype
- Function Call
- Function Definition
- Passing Arguments to a Function

In next lecture we will discuss:

- Inline functions
- Function overloading
- Function default values
- Templates
- Storage classes